



# Testes em ambientes ágeis

**E**m ambientes ágeis, os testes de software são fundamentais para assegurar qualidade contínua e entregas incrementais, sendo integrados desde as primeiras etapas do desenvolvimento.

Diferente dos métodos tradicionais, onde os testes ocorrem apenas no final do desenvolvimento, no desenvolvimento ágil, os testes são contínuos, colaborativos e automatizados.

Os testes em ambientes ágeis são uma abordagem integrada ao ciclo de desenvolvimento, onde testadores, desenvolvedores e stakeholders trabalham juntos para garantir a qualidade do software desde o início.

| Característica   | Testes Tradicionais         | Testes Ágeis                       |
|------------------|-----------------------------|------------------------------------|
| Momento do Teste | No final do desenvolvimento | Durante todo o processo            |
| Responsável      | Equipe de testes separada   | Desenvolvedores + Testadores       |
| Entrega          | Lotes grandes e demorados   | Entregas frequentes e incrementais |
| Automação        | Opcional                    | Fortemente incentivada             |

Em metodologias como Scrum e Kanban, os testes fazem parte do fluxo de trabalho contínuo. Eles garantem que cada incremento do software esteja funcional e livre de erros.

- Testes ocorrem a cada Sprint
- Testadores trabalham junto com desenvolvedores
- Automação ajuda a manter a qualidade

No contexto ágil, a aplicação de testes constantes reforça a colaboração entre equipes e a entrega de software funcional. As equipes de desenvolvimento, testes e stakeholders colaboram de forma contínua para garantir que o produto atenda aos requisitos e expectativas do cliente. Aqui estão algumas das principais práticas de teste em metodologias ágeis.

A principal característica das práticas ágeis é o teste contínuo, onde os testes são realizados ao longo de todo o ciclo de desenvolvimento, desde o início até a entrega.

- **Testes são executados em cada iteração (Sprint).**
- **Integração contínua (CI) e entrega contínua (CD)**
- Garante que o código seja testado frequentemente.
- **Feedback rápido**
- Identifica e corrige os problemas cedo.

#### **Exemplo:**

Sempre que um desenvolvedor faz uma alteração no código, o pipeline de CI executa os testes automatizados para garantir que as alterações não quebraram nenhuma funcionalidade existente.

#### **Colaboração entre Testadores e Desenvolvedores**

Em ambientes ágeis, **testadores e desenvolvedores trabalham juntos** desde o início do projeto para garantir que os testes atendam aos requisitos do cliente e aos critérios de aceitação.

- **Testes são planejados e executados de forma colaborativa.**
- **Testadores ajudam a criar critérios de aceitação e testes automatizados.**
- **Desenvolvedores escrevem testes unitários**
- Garante a qualidade do código desde o início.

#### **Exemplo:**

Durante uma Sprint, o testador e o desenvolvedor discutem as histórias de usuário e definem os **cenários de teste** juntos.

#### **Testes Baseados em Comportamento (BDD)**

No ágil, o Test Driven Development (TDD) e o Behavior Driven Development (BDD) são usados para promover uma abordagem focada no comportamento do sistema e do usuário. O BDD utiliza uma linguagem comum que pode ser entendida tanto por desenvolvedores quanto por não-técnicos, facilitando a colaboração.

- **Especificação de comportamento**

- Forma legível por todos os stakeholders.
- **Exemplos de comportamento**
- Descritos em linguagem natural.
- Ferramentas como **Cucumber** e **SpecFlow**
- Permite escrever testes de comportamento de forma colaborativa.

### **Exemplo em Gherkin (Cucumber):**

Feature: Login no sistema

Scenario: Login com credenciais válidas

Given que o usuário está na tela de login

When ele insere um e-mail válido e senha

Then ele deve ser redirecionado para o dashboard

### **Testes Automatizados**

A automação de testes é essencial para garantir que os testes sejam executados de forma rápida e contínua. Ela permite que os testes sejam repetidos várias vezes sem a necessidade de intervenção manual, aumentando a cobertura e a eficiência.

- **Testes automatizados são executados durante o processo de integração contínua (CI/CD).**

- **Testes unitários, de integração e de aceitação**

· São testes frequentemente automatizados.

- **Automatizar testes de regressão**

· Ajudam a manter a qualidade enquanto novas funcionalidades são adicionadas.

### **Exemplo:**

Testes de interface de usuário podem ser automatizados com ferramentas como **Selenium** ou **Cypress**, enquanto testes de API podem ser feitos com **Postman**.

### **Testes de Regressão Rápidos**

Em ambientes ágeis, os testes de regressão garantem que novas funcionalidades não quebrem as existentes. Como o código é alterado

frequentemente, esses testes devem ser rápidos e automatizados para não impactar a velocidade do desenvolvimento.

- **Testes de regressão são realizados com cada nova entrega**

- Geralmente no final de cada Sprint.

- **Automatização**

- Prática chave para esses testes.

- **A cobertura de testes deve ser expandida conforme o sistema cresce.**

### **Exemplo:**

Toda vez que uma nova funcionalidade é implementada, o sistema é testado para garantir que as funcionalidades existentes (como login ou cadastro de usuário) funcionem corretamente.

### **Testes Baseados em Histórias de Usuário e Critérios de Aceitação**

As **histórias de usuário** e **critérios de aceitação** são a base para os testes em ambientes ágeis. Elas ajudam a definir claramente o que precisa ser testado.

- **Cenários de testes são derivados diretamente das histórias de usuário.**

- **Critérios de aceitação**

- Detalham as expectativas do cliente para cada funcionalidade.

- **Testes funcionais e de aceitação**

- Testes alinhados com os critérios pré-definidos.

### **Exemplo de História de Usuário:**

**Como** usuário, **quero** poder buscar produtos no e-commerce, **para que** eu possa encontrar o que desejo comprar.

### **Critério de Aceitação:**

- O usuário deve ser capaz de digitar um termo de busca e ver uma lista de produtos relacionados.

- A busca deve funcionar com filtros (por preço, categoria, etc.).

### **Testes Exploratórios**

Embora os testes ágeis incentivem a automação, os testes exploratórios também são importantes. Eles são realizados de forma ad-hoc, sem scripts

predefinidos, para descobrir defeitos inesperados.

- **Testadores exploram o sistema de forma criativa.**
- **Focado na descoberta de problemas não previstos.**
- **Geralmente realizados no final de uma Sprint**, antes da entrega.

#### **Exemplo:**

Testar uma nova funcionalidade sem seguir um roteiro, simplesmente interagindo com a interface para verificar seu comportamento inesperado.

### **TDD e BDD: Testes Orientados a Desenvolvimento e Comportamento**

**TDD (Test-Driven Development)** e **BDD (Behavior-Driven Development)** são duas abordagens populares no desenvolvimento ágil que ajudam a garantir a qualidade do software. Embora ambos envolvam escrever testes antes do código, eles têm enfoques diferentes. Vamos entender o que são, como funcionam e suas diferenças.

#### **O que é TDD?**

**Test-Driven Development (TDD)** é uma abordagem de desenvolvimento em que os **testes são escritos antes do código**. A ideia central do TDD é que os desenvolvedores escrevem um **teste automatizado** que falha inicialmente (porque o código ainda não foi implementado) e, então, escrevem o código para fazer esse teste passar.

#### **Ciclo de TDD**

O processo de TDD segue um ciclo curto e repetitivo, conhecido como **Red-Green-Refactor**:

- 1. Red:** Escreve-se um teste que falha, porque a funcionalidade ainda não foi implementada.
- 2. Green:** Escreve-se o código mínimo necessário para passar no teste.
- 3. Refactor:** Refatora-se o código para melhorar sua qualidade, sem quebrar o teste.

#### **Benefícios do TDD**

- **Código mais testável e modular.**
- **Detecção precoce de falhas.**

- **Melhor design de código**

- Foco em soluções pequenas e testáveis.

- **Documentação automática,**

- Testes servem como uma documentação do comportamento do código.

## O que é BDD?

**Behavior-Driven Development (BDD)** é uma abordagem de desenvolvimento que enfatiza a **colaboração entre desenvolvedores, testadores e stakeholders** (usuários, clientes) para definir o comportamento desejado de uma aplicação. BDD foca em **como o sistema deve se comportar em diferentes cenários**, com uma linguagem comum e compreensível para todos os envolvidos.

## Características do BDD

- **Linguagem Simples e Comum:** Usa uma sintaxe comum, como o **Gherkin**, que é entendida tanto por técnicos quanto não-técnicos.

- **Cenários de Comportamento:** São descritos em termos de **“Given-When-Then”** (Dado-Quando-Então), o que ajuda a focar no comportamento do sistema.

- **Testes são escritos como exemplos concretos** do comportamento que o software deve ter.

## Sintaxe Gherkin

A sintaxe do Gherkin permite descrever o comportamento da aplicação de forma simples e estruturada:

- **Given:** Contexto inicial ou estado do sistema.

- **When:** A ação ou evento que ocorre.

- **Then:** O resultado esperado desta ação.

## Exemplo de Cenário BDD em Gherkin:

Feature: Login de usuário

Scenario: Login com credenciais válidas

Given que o usuário está na página de login

When ele insere um e-mail e senha válidos

Then ele deve ser redirecionado para a página inicial

## Ferramentas de BDD

- **Cucumber:** Uma das ferramentas mais populares para BDD, que permite escrever testes de comportamento com Gherkin.
- **SpecFlow:** Similar ao Cucumber, mas voltado para .NET.
- **Behat:** Uma ferramenta de BDD para PHP.

A **cobertura de testes** mede o **quanto do código foi executado** durante os testes automatizados. Essa métrica é usada para avaliar a eficácia dos testes e a **probabilidade de falhas** no sistema.

A cobertura de testes pode ser expressa em **porcentagem**. Por exemplo, se uma aplicação tem 80% de cobertura de testes, isso significa que 80% do código foi executado e testado, enquanto os 20% restantes não foram cobertos pelos testes.

Existem vários tipos de cobertura de testes, cada uma abordando uma parte diferente do código.

### 1. Cobertura de Linha (Line Coverage)

- Mede a porcentagem de **linhas de código** que foram executadas durante os testes.
- A ideia é que, se uma linha de código for executada por um teste, ela foi “coberta”.
- **Limitação:** Não garante que todas as **possíveis execuções** do código sejam testadas.
- **Exemplo:** Se o código tiver 100 linhas e 80 dessas linhas forem executadas em seus testes, a cobertura de linha será de **80%**.

### 2. Cobertura de Bloco (Block Coverage)

- Verifica se todos os **blocos de código** (seções de código delimitadas por estruturas de controle como loops ou condicionais) foram executados.

### 3. Cobertura de Condição (Condition Coverage)

- Avalia se todas as **condições lógicas** dentro de estruturas de controle (como if, else, while) foram avaliadas tanto para verdadeiro quanto para falso.
- **Exemplo:** Em um código com um if (a > b && c < d), a cobertura de

condição garante que os testes verifiquem ambos os **casos verdadeiros e falsos** para as condições.

#### 4. Cobertura de Caminho (Path Coverage)

- Verifica se todas as **sequências possíveis de execução** do código foram testadas.
- Isso inclui combinações de decisões que podem levar a diferentes caminhos de execução no código.
- **Exemplo:** Se um código tiver várias instruções condicionais, a cobertura de caminho assegura que todos os possíveis fluxos de execução sejam testados.

#### 5. Cobertura de Função (Function Coverage)

- Mede a quantidade de **funções/métodos** que foram chamadas durante os testes.
- A cobertura de função verifica se todas as funções que o sistema oferece estão sendo invocadas por algum teste.

#### Ferramentas para Medir Cobertura de Testes

Existem várias ferramentas que podem ser usadas para **analisar e medir a cobertura de testes**. Algumas das mais populares incluem:

- **JaCoCo** (Java): Ferramenta popular para medir a cobertura de código em aplicações Java.
- **Istanbul** (JavaScript): Biblioteca para medir a cobertura de testes em projetos Node.js.
- **Clover** (Java, Groovy): Ferramenta de cobertura para projetos Java e Groovy.
- **pytest-cov** (Python): Plugin para o PyTest que fornece informações sobre a cobertura de testes em Python.
- **Cobertura** (Java): Ferramenta de código aberto para medir a cobertura de testes em Java.
- **Codecov**: Plataforma de análise de cobertura que pode ser usada para diversos tipos de código e integração com serviços de CI/CD.

**Testes exploratórios** são uma abordagem de teste onde o testador **explora ativamente o sistema** sem seguir um roteiro predefinido. O objetivo é



descobrir comportamentos inesperados, falhas e aspectos que não foram antecipados durante o planejamento formal. Diferente de testes baseados em scripts, os testes exploratórios dependem da **criatividade** e do **conhecimento do sistema** por parte do testador.

Os **testes exploratórios** são uma forma de **testar software de maneira não estruturada**, onde o testador não segue um conjunto rígido de passos. Em vez disso, ele usa **intuição, conhecimento prévio e experimentação** para encontrar falhas. Esse tipo de teste é **dinâmico e adaptativo**, permitindo ao testador **explorar funcionalidades** de maneiras novas e inesperadas.

#### **Características principais:**

- **Adaptação em tempo real:** O testador muda a abordagem conforme novas informações ou falhas são encontradas.
- **Foco no comportamento do sistema:** Explorar como o sistema reage a diferentes entradas e interações.
- **Testes de áreas não documentadas ou não especificadas:** Os testes exploratórios frequentemente ajudam a identificar falhas em funcionalidades que não foram cobertas nos casos de teste tradicionais.
- **Feedback rápido e eficiente:** Ideal para encontrar falhas rápidas durante o ciclo de desenvolvimento ágil.

### **Como Realizar Testes Exploratórios?**

#### **1. Preparação Inicial**

Embora os testes exploratórios não sigam um roteiro fixo, é importante realizar uma **preparação inicial**:

- **Entenda o Sistema:** Tenha uma boa compreensão do sistema que será testado. Leia a documentação, se disponível, e converse com desenvolvedores ou stakeholders sobre as funcionalidades principais.
- **Identifique Áreas Críticas:** Baseie-se em riscos, funcionalidades críticas ou áreas recém-modificadas do sistema que merecem atenção.
- **Defina Limites:** Mesmo que o teste seja exploratório, defina limites claros, como os recursos a serem explorados, o tempo de execução e os objetivos esperados.

## 2. Técnicas de Testes Exploratórios

Existem diversas técnicas que podem ser usadas para **maximizar a eficácia** dos testes exploratórios:

### 1. Heurísticas de Testes

Utilizar heurísticas de teste pode ajudar a conduzir a exploração. Heurísticas são regras práticas que orientam o testador, mas sem garantir uma solução definitiva. Exemplos incluem:

- **Testar entradas válidas e inválidas.**
- **Testar limites de valores.**
- **Testar interações entre funcionalidades.**
- **Alterar sequências de ações.**

### 2. Sessões de Teste

As **sessões de teste** envolvem um período dedicado onde o testador foca completamente em explorar o sistema. Durante essa sessão, o testador pode adotar diferentes abordagens, como:

- **Explorar uma funcionalidade específica** (por exemplo, uma tela de login ou um fluxo de pagamento).
- **Testar como o sistema reage a erros** ou situações inesperadas.
- **Realizar testes de desempenho simples** (ex.: clicar rapidamente em botões ou enviar múltiplos formulários simultaneamente).

### 3. Testes Baseados em Características

Foque em características específicas do sistema, como:

- **Testar a interface de usuário (UI):** Explore diferentes combinações de entrada de dados, clique em botões, altere configurações e veja como o sistema reage.
- **Testar a segurança:** Tente acessar funcionalidades com permissões inadequadas ou simule ataques comuns, como injeção de SQL.
- **Testar a usabilidade:** Experimente funcionalidades para verificar a experiência do usuário e como a interface pode ser intuitiva ou confusa.

### 3. Registro Durante os Testes

Durante os testes exploratórios, é fundamental **registrar** os passos tomados, falhas encontradas e insights importantes. Algumas dicas

incluem:

- **Documentar as ações:** Embora o processo seja exploratório, registre o que foi testado, como foi testado e o que funcionou ou não.
- **Tire capturas de tela ou grave vídeos:** Caso uma falha importante seja encontrada, isso pode ser útil para os desenvolvedores.
- **Anotar hipóteses:** Durante o processo, o testador pode fazer suposições ou previsões sobre o comportamento do sistema. Documentar essas hipóteses pode ser útil para testar mais a fundo.

## Vantagens dos Testes Exploratórios

### 1. Detecção Rápida de Defeitos

Testes exploratórios são rápidos e eficientes, sendo ideais para encontrar **defeitos inesperados** que podem não ser cobertos por testes baseados em casos predefinidos.

### 2. Flexibilidade

A abordagem é muito flexível. O testador pode mudar rapidamente de foco, explorar novas áreas ou reagir de acordo com os resultados encontrados, sem estar preso a um roteiro rígido.

### 3. Teste de Funcionalidades Não Especificadas

É útil para **testar funcionalidades novas ou não documentadas**, permitindo descobrir problemas que podem não ter sido previstos ou especificados.

### 4. Identificação de Problemas de Usabilidade

Testes exploratórios são eficazes para encontrar problemas de **usabilidade** e **experiência do usuário (UX)**, algo que pode ser difícil de prever com scripts de teste tradicionais.

## GitHub da Disciplina

Confira os códigos e scripts usados ao longo da disciplina no repositório dela no GitHub: <https://github.com/FaculdadeDescomplica/Pratica-Integradora-com-Metodos-Ageis>

## Conteúdo Bônus

Um documentário interessante para assistir sobre o assunto de ágil, podemos citar o “Agile: The Movie” um documentário disponível no Youtube que mostra o dia a dia de uma equipe de Scrum fictícia, incluindo práticas de testes, com foco nos papéis no Scrum, histórias de usuário, colaboração e testes ágeis. Um outro documentário interessante que introduz o devops e como testes automatizados se integram ao ciclo ágil é o “DevOps: The Big Picture”; ele foca em entrega contínua, integração de testes e cultura colaborativa.

## Referências Bibliográficas

BACH, James. Exploratory Testing Explained. 2003.

FOWLER, Martin. Test Coverage. 2012.

WYNNE, Matt; HELLESOY, Aslak. The Cucumber Book: Behaviour-Driven Development for Testers and Developers. Raleigh: Pragmatic Bookshelf, 2012.

Você sabe o que é cobertura de testes (TDD)? Leonardo Neto, outubro de 2023. Disponível em <https://www.dio.me/articles/voce-sabe-o-que-e-cobertura-de-testes-tdd>. Acesso em 23 de abril de 2025.

Um pouco sobre cobertura de código e cobertura de testes. Alex Candido, 08 de outubro de 2019. Disponível em <https://medium.com/liferay-engineering-brazil/um-pouco-sobre-cobertura-de-código-e-cobertura-de-testes-4fd062e91007>. Acesso em 23 de abril de 2025.

Test Driven Development: TDD Simples e Prático. DevMedia. Disponível em <https://www.devmedia.com.br/test-driven-development-tdd-simples-e-pratico/18533>. Acesso em 23 de abril de 2025.

O que é TDD? Beatriz Feliciano, 20 de julho de 2021. Disponível em <https://dev.to/womakerscode/o-que-e-tdd-4b5f>. Acesso em 23 de abril de 2025.

Behavior Driven Development (BDD): saiba como aplicar, quais as fases e os benefícios. Objective. Disponível em <https://www.objective.com.br/insights/bdd/#:~:text=O Behavior Driven>

**Development (BDD)** é uma técnica de desenvolvimento,positiva para o usuário final. Acesso em 23 de abril de 2025.

Desenvolvimento orientado por comportamento (BDD). DevMedia. Disponível em <https://www.devmedia.com.br/desenvolvimento-orientado-por-comportamento-bdd/21127>. Acesso em 23 de abril de 2025.

## **Atividade Prática 9 – Testes em ambientes ágeis**

**Título da Prática:** Testes em ambientes ágeis

**Objetivos:** Compreender e aplicar práticas de testes contínuos, TDD e BDD no contexto de metodologias ágeis.

**Materiais, Métodos e Ferramentas:** Para realizar esta prática vamos ler atentamente o conteúdo da unidade

### **Atividade Prática**

**1.** Durante uma sprint, a equipe de desenvolvimento percebe que muitas funcionalidades entregues possuem falhas, atrasando a entrega ao cliente. O Product Owner sugeriu que fossem adotadas práticas de teste típicas de ambientes ágeis para melhorar a qualidade e evitar retrabalho nas próximas iterações.

Cite e explique pelo menos três práticas comuns de teste em ambientes ágeis e justifique como cada uma delas pode ajudar a melhorar a qualidade das entregas.

**2.** Uma equipe deseja melhorar a qualidade do seu código e evitar que funcionalidades sejam implementadas sem estarem bem definidas. O líder técnico propôs a adoção de TDD e BDD como práticas de desenvolvimento orientadas a testes.

Explique o que são TDD (Test-Driven Development) e BDD (Behavior-Driven Development), destacando as diferenças entre eles. Cite uma vantagem de cada prática.

### **Gabarito Atividade Prática**

**1.** Três práticas comuns de teste em ágil:

### Automatização de testes:

Automatizar testes de regressão, unidade e integração para garantir que funcionalidades antigas não sejam quebradas por novas implementações. Aumenta a confiabilidade e reduz o tempo gasto com testes manuais.

### Testes contínuos:

Realizar testes em todas as fases do desenvolvimento, integrando-os ao pipeline de integração contínua (CI). Garante que falhas sejam detectadas rapidamente, evitando o acúmulo de problemas.

### Reuniões de definição de critérios de aceitação (ATDD):

Colaborar com o Product Owner para definir claramente os critérios de aceitação antes do desenvolvimento. Garante que todos entendam o que é necessário entregar, reduzindo a chance de desenvolver funcionalidades incorretas.

## 2. TDD - Test-Driven Development:

É uma prática de desenvolvimento onde os testes são escritos antes da implementação do código, guiando o design e assegurando que as funcionalidades atendam aos requisitos técnicos.

Vantagem: Garante código testável e com menos defeitos, já que os testes orientam a implementação desde o início.

### BDD - Behavior-Driven Development:

É uma evolução do TDD, focada na definição e automação de testes a partir do comportamento esperado do sistema, usando linguagem natural, compreensível por todas as partes interessadas.

Vantagem: Melhora a comunicação entre desenvolvedores, testers e stakeholders, assegurando que o software atenda às necessidades de negócio.

### Diferença principal:

TDD foca em testes de unidade e na qualidade técnica do código, enquanto BDD foca no comportamento do sistema como um todo, promovendo alinhamento entre as áreas técnicas e de negócio. Comentário: A resposta deve explicar cada prática, apontar diferenças e apresentar vantagens aplicáveis ao contexto.

## **Ir para exercício**